TPSI · STRUMENTI DI SVILUPPO CON L'INTELLIGENZA ARTIFICIALE

Claude Code

Un assistente AI che non si limita a rispondere: legge il progetto, modifica i file ed esegue i comandi. Come funziona e come usarlo con giudizio.

IL PUNTO DI PARTENZA

Da chi risponde a chi agisce

Chatbot

- Vede solo ciò che incolli nella chat.
- Ti suggerisce il codice.
- Copi, incolli, compili e provi **tu**.

Agente

Esplora da solo i file del progetto.

Modifica direttamente il codice.

Esegue compilazione e test, legge gli errori e corregge.

Un **agente** è un modello di linguaggio che può usare strumenti e decidere da solo il passo successivo.

CHE COS'È

Claude Code in breve

- Strumento di programmazione **agentico** sviluppato da Anthropic.
- Lavora dentro la cartella del tuo progetto.
- Legge e scrive file, esegue comandi da terminale, usa `git`.
- Si comanda in linguaggio naturale, anche in italiano.
- Richiede un account Claude o una chiave API.

Dove si usa

Terminale · VS Code · JetBrains

App desktop · Browser

COME RAGIONA

Il ciclo dell'agente

1

Capisce

Legge la richiesta e i file utili del progetto.

2

Pianifica

Decide quali file cambiare e in che ordine.

3

Agisce

Modifica il codice ed esegue i comandi.

4

Verifica

Compila, lancia i test, legge gli errori.

Se la verifica fallisce, il ciclo **ricomincia**: è lo stesso modo di lavorare di un buon programmatore.

UN ESEMPIO

Una richiesta, tanti passi

```
$ claude
> Il server della chat si blocca quando un client si disconnette.
  Trova la causa, correggila e verifica con un test.

● Leggo ServerChat.java e GestoreClient.java
● Causa: l'eccezione di readLine() non viene gestita
● Modifico GestoreClient.java   ← chiede il permesso
● Eseguo i test: 3 superati, 0 falliti
```

Esempio illustrativo: l'aspetto reale dello schermo cambia con la versione del programma.

CHI DECIDE

Tu resti al comando

?

Permessi

Prima di modificare file o eseguire comandi chiede conferma. Puoi accettare, rifiutare o correggere la rotta.

≡

Modalità piano

Con **Maiusc + Tab** si passa alla modalità piano: l'agente analizza e propone, senza toccare nulla.

■

Interruzione

Con **Esc** lo fermi in qualsiasi momento e gli dai nuove indicazioni.

Buona abitudine: chiedi prima il **piano**, leggilo, e solo dopo lascia che scriva il codice.

AL COMPUTER · 15 minuti

Scrivi una buona richiesta

Questa richiesta è troppo vaga: **"sistema il login del sito"**.

Apri un editor di testo e riscrivila in un file `richiesta.txt` con quattro parti:

1. **Contesto**: che progetto è, quali file.
2. **Problema**: cosa succede ora.
3. **Obiettivo**: cosa deve succedere.
4. **Verifica**: come capire che funziona.

Suggerimento

Immagina di scrivere a un collega bravo che non ha mai visto il tuo progetto.

Controllo: scambia il file con il compagno. Riesce a capire cosa fare senza farti domande?

LA MEMORIA DEL PROGETTO

Il file CLAUDE.md

Un file di testo nella cartella del progetto che l'agente **legge all'inizio di ogni sessione**.

- Come si compila e come si lanciano i test.
- Regole di stile del codice.
- Cose da non fare.

Il comando `/init` ne crea una prima versione analizzando il progetto.

```
# Chat TCP in Java

## Comandi
- Compila: javac -d out src/*.java
- Server:  java -cp out ServerChat

## Regole
- Nomi di classi e metodi in italiano
- Porta di default: 5000
- Non modificare Protocollo.java
```

AL COMPUTER · 20 minuti

Il CLAUDE.md del tuo progetto

Scegli un progetto che hai già sviluppato in TPSI (ad esempio il client/server con i socket). Nella sua cartella crea il file `CLAUDE.md` con:

- una riga che descrive il progetto;
- i comandi esatti per compilare ed eseguire;
- almeno tre regole del progetto;
- almeno una cosa da **non** fare.

Prova sul campo

Apri un terminale nuovo e copia i tuoi comandi uno a uno: devono funzionare così come sono scritti.

Controllo: un compagno, leggendo solo il tuo file, riesce a compilare e avviare il progetto.

COMANDI DA CONOSCERE

Scorciatoie della sessione

Comando	A cosa serve
`/init`	Crea un primo `CLAUDE.md` dal progetto
`/clear`	Ricomincia da zero, dimenticando la conversazione
`/compact`	Riassume la conversazione per liberare spazio
`/help`	Elenca tutti i comandi disponibili
`Maiusc + Tab`	Cambia modalità di permesso, compresa la modalità piano
`Esc`	Interrompe l'agente

OLTRE LE BASI

Si può estendere

MCP

Strumenti esterni

Il Model Context Protocol è uno standard aperto per collegare l'agente a database, GitHub, documentazione e altri servizi.

Sub

Subagenti

Agenti specializzati a cui delegare un compito, ad esempio cercare nel codice o fare una revisione.

Hook

Automatismi

Comandi che partono da soli in certi momenti, ad esempio formattare il codice dopo ogni modifica.

Idea chiave: l'agente diventa più utile quanto più contesto e strumenti gli dai.

RICORDATELO SEMPRE

L'AI scrive il codice.
La responsabilità resta tua.

AL COMPUTER · 15 minuti

Revisiona il codice generato

Un agente ha scritto questo metodo per calcolare la media dei voti. Copialo in un progetto Java e provalo con `{6, 7, 8}`.

```
public static double media(int[] voti) {
    int somma = 0;
    for (int i = 1; i < voti.length; i++) {
        somma += voti[i];
    }
    return somma / voti.length;
}
```

Consegna

Trova gli errori, correggili e scrivi in un commento perché erano sbagliati.

Controllo: ci sono almeno due errori. Corretto, il metodo restituisce **7.0**; com'è ora restituisce 5.0.

USARLO BENE

Buone pratiche e rischi

Da fare

- Usa `git` e fai commit spesso: puoi sempre tornare indietro.
- Chiedi il piano prima del codice.
- Leggi ogni modifica e lancia i test.

Da evitare

- Inserire password, chiavi o dati personali.
- Accettare comandi che non capisci.
- Consegnare codice che non sai spiegare.

RIPASSO

Le tre cose da ricordare

1

Agente

Capisce, pianifica, agisce e verifica dentro il tuo progetto.

2

Contesto

Richieste precise e un buon `CLAUDE.md` fanno la differenza.

3

Controllo

Permessi, piano, revisione e test: il responsabile sei tu.